The Open Source Audit

A Capstone Project for OSS Course

Kirti Sonam Pattanik
Registration Number:
24BAI10611
Chosen Software: Linux Kernel

March 2026

1 Introduction

The Linux kernel is a piece of software which sits between Hardware and the OS, responsible for managing tasks such as PAGING and POOLING , ZRAM etc.

At it's core Linux Kernel is free and open source unlike Microsoft's MS-DOS / Windows Kernel

It was created in 1991 by Linus Trovold

2 Part A — Origin and Philosophy

2.1 A1. Unix was expensive and not easily affordable

UNIX limitations:

While UNIX distinguished itself from its predecessors as the first portable operating system which was written entirely in C , it had some major limitations:

- Unix was CLI based and had not GUI interface
- The learning curve was super steep as compared to other OS at the time
- Not beginner friendly at all

Story of Linus Torvalds

Linus Torvalds was born in Helsinki, Finland, on 28 December 1969. His parents were campus radicals at the University of Helsinki in the 1960s. His family belongs to the Swedish-speaking minority in Finland. His interest in Computer Science started at the ripe age of 11 when he started programming in BASIC and then later went onto access CPU via machine code directly instead of assembly

Why Linux was created

The Linux kernel was created by Linus Torvalds, He made it because there was a lack of working kernel for GNU / OPEN SOURCE SOFTWARES, a working open source version of the UNIX version made entierly out of open sourced free softwares.

It was originally made for x86 based computers but because of the community effort it's now responsible for the ANDROID and IOS ecosystems as they have evolved from the linux kernels

2.2 A2. The License — What It Actually Says

The Linux Kernel is provided under: **SPDX-License-Identifier: GPL-2.0**

While most of the files for the kernel are under: **GNU/GPL (General Public License V2)**

With an explicit syscall exception, as stated at: **LICENSES/exceptions/Linux-syscall-note**
In the Linux Kernel Github

GNU license enables the community to use the parts of Linux kernel outside of the main ecosystem / the main linux kernel file

2.3 A3. The Ethics of Open Source

- Open-source software raises ethical questions around transparency, accountability, and community responsibility. By design, open-source projects allow anyone to view, modify, and distribute code, which promotes collaboration and innovation. However, this openness also means maintainers have a duty to ensure their code is secure and well-documented.
- Another key consideration is inclusivity and fair contribution. Open-source communities often depend on unpaid labor, which can lead to burnout among maintainers and exclude contributors from underrepresented groups.

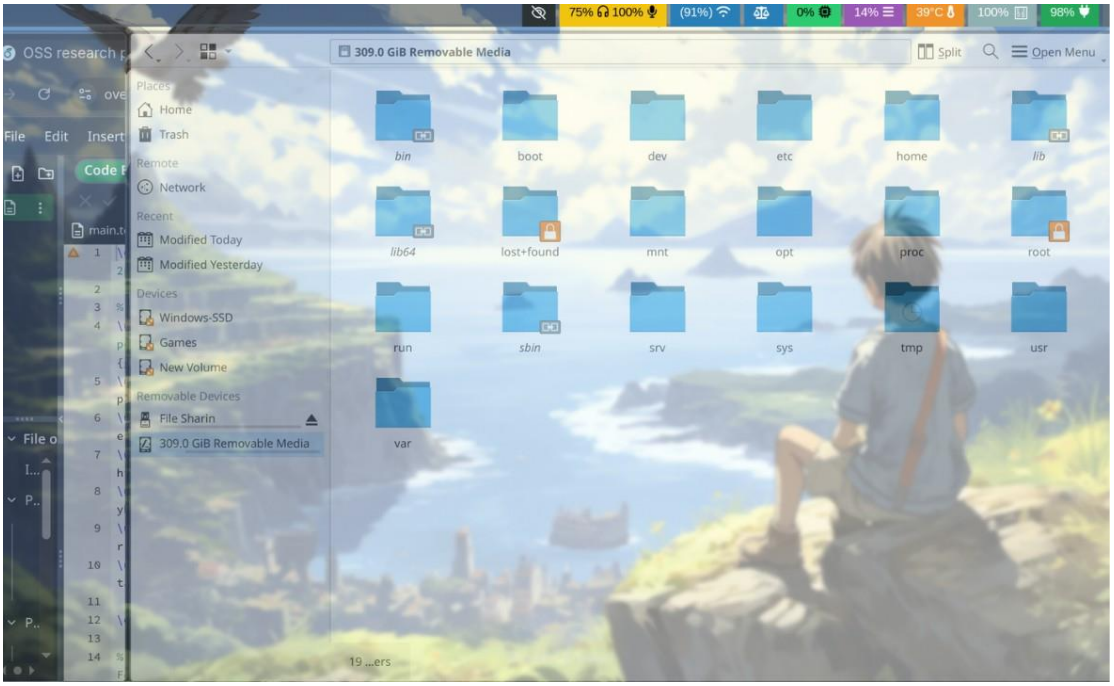
3 Part B — Linux Footprint

- Installing Packages on linux depend from architecture to architecture , for this report we will be using **ARCH LINUX**.
- In **ARCH LINUX** , packages are installed via: **PACMAN** package manager with the command: **sudo pacman -S Package Name**
- In linux there can be multiple users like in windows but the MAIN user / ADMIN of the system is the **ROOT** user in order to make a new user follow the following steps:

  sudo useradd -m USERNAME
  Then to set a password we use: **passwd USERNAME**
- Some of the core directories of linux are:
  - Root Directory (/): This directory is the Root entry point of the entire file system
  - Binaries (/bin): This directory contains the user-essential files necessary to run the commands like: **ls**,**cp**,**mv**
  - System binaries (/sbin): This directory contains the System Level binary files which are responsible for commands like: **fsck**,**reboot**,**iptables**

- User files (/usr): Contains the files installed by the user for softwares like VLC , Chromium , etc.
- Home directory (/home): Each user get's a personal Directory here say if you have 2 users on a system (Admin) and (Guest) they will have /home/Admin and /home/Guest
- ETC directory (/etc): Contains system wide config files which do not depend from user to user
- Variables (/var): Files in this directory keep changing such as LOGS , TEMP files and databases
- Boot (/boot): Contains boot loader and kernel files necessary to boot up linux File System.
- dev (/dev): Represents hardware as files such as /dev/sda -¿ hard drive
- processes (/proc): Stores FileSystem and Kernel information

Screenshot



4 Part C — The FOSS Ecosystem

- The Free and Open Source Software (FOSS) ecosystem is built upon a strong foundation of shared tools, collaborative development, and transparent governance. One of its core strengths lies in its dependency chain, where projects rely on essential components such as the GNU Compiler Collection (GCC) and various GNU utilities. These tools provide the fundamental building blocks required to compile, debug,

and run software across different platforms, ensuring consistency and portability within the ecosystem.

- Linux plays a central role in enabling a wide range of technologies and platforms. It serves as the backbone of modern server infrastructure, powering a significant portion of the internet due to its stability, scalability, and security. Additionally, Linux forms the core of the Android operating system, which dominates the global mobile market. This demonstrates how open-source software can scale from low-level system components to widely used consumer technologies.

- Another defining aspect of the FOSS ecosystem is its community-driven nature and governance model. Projects are typically maintained by a global network of contributors, including individual developers, organizations, and foundations. Decision-making processes are often transparent, with contributions reviewed and discussed openly. This collaborative environment encourages innovation, rapid problem-solving, and continuous improvement, while also ensuring that the software remains accessible and adaptable to the needs of its users.

5 Part D — Open Source vs Proprietary

Dimension	**Linux Kernel (Open Source)**	**Windows Kernel (Proprietary)**
Cost	Free and open source	Locked and closed source
Security	More than windows	Less Secured
Support	Depends from distro to distro	Centralized and unified
Freedom to Modify	Yes	No
Control	Full	Partial

Conclusion

- Linux provides High System level access due it's open source nature , allowing user's to make their own OS from scratch if they have the knowledge on how to. A perfect example of this is LFS (Linux From Scratch).

- In contrast Windows is Closed source and Propritery OS managed by Microsoft offering more standardized and centralized way of support.

- While windows is generally considered stable , efforts from **Valve Corporation** have been pushing Linux to more stable version with it's **Proton** compatibility layer

and **Steam OS** to be a better option for Gaming resulting in **MASS** adoption of the Linux Ecosystem for the general public

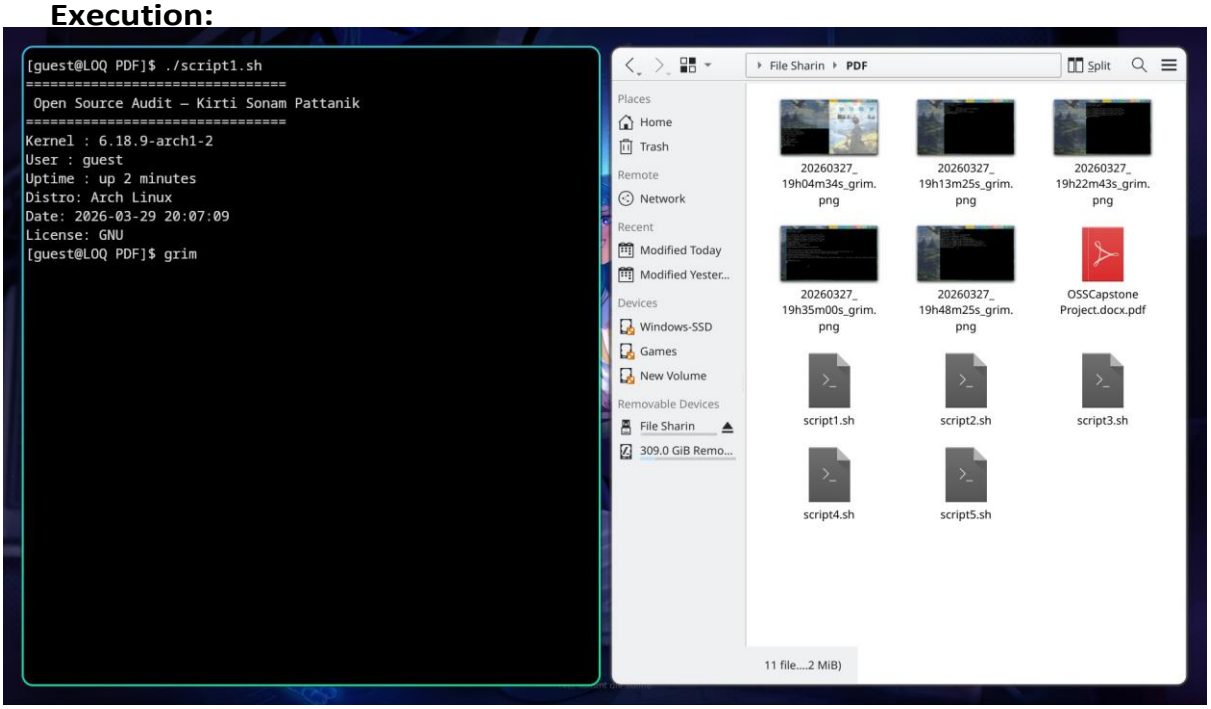
6 Shell Scripts

6.1 Script 1 — System Identity Report

Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Course: Open Source Software
# --- Variables ---
STUDENT_NAME="Kirti Sonam Pattanik" # Fill in your name
SOFTWARE_CHOICE="Linux Kernel" # Fill in your chosen software
# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
# --- Display ---
echo "================================"
echo " Open Source Audit | $STUDENT_NAME"
echo "================================"
echo "Kernel : $KERNEL"
echo "User : $USER_NAME"
echo "Uptime : $UPTIME"
. /etc/os-release
echo "Distro: $NAME"
echo "Date: $(date +"%Y-%m-%d %H:%M:%S")"
echo "License: GNU"
# TODO: Add distro name, date, and license message
```

Execution:



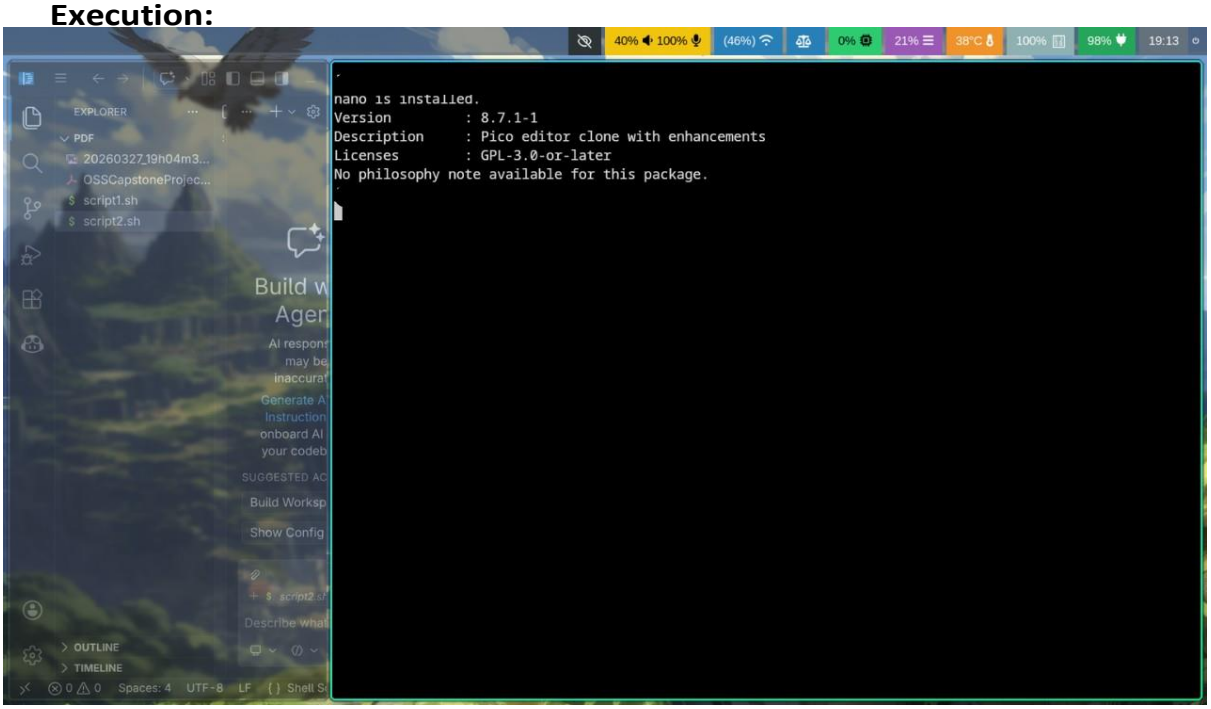
Explanation: The script first Stores Student name into a variable and then software choice , after which it dynamically fetches system info from the system such as KERNEL , Current USERNAME and System UPTIME and then displays all that info in the terminal

6.2 Script 2 — FOSS Package Inspector

```
#!/bin/bash
# Script 2: FOSS Package Inspector (Arch Linux)

PACKAGE="nano" # take package as argument

if [ -z "$PACKAGE" ]; then
    echo "Usage: $0 <package-name>"
    exit 1
fi

# Check if package is installed
if pacman -Q "$PACKAGE" &>/dev/null; then
    echo "$PACKAGE is installed."

    # Show package info (Version, License, Description)
    pacman -Qi "$PACKAGE" | grep -E 'Version|Licenses|Description'
else
    echo "$PACKAGE is NOT installed."
```

```
    exit 0
fi

# Philosophy notes
case "$PACKAGE" in
    apache | httpd)
        echo "Apache: the web server that built the open internet"
        ;;
    mysql)
        echo "MySQL: open source at the heart of millions of apps"
        ;;
    nginx)
        echo "Nginx: performance-focused minimalism for the modern web"
        ;;
    firefox)
        echo "Firefox: privacy-first browsing in a corporate web"
        ;;
    vlc)
        echo "VLC: if it exists, it plays it"
        ;;
    *)
        echo "No philosophy note available for this package."
        ;;
esac
```

Execution:

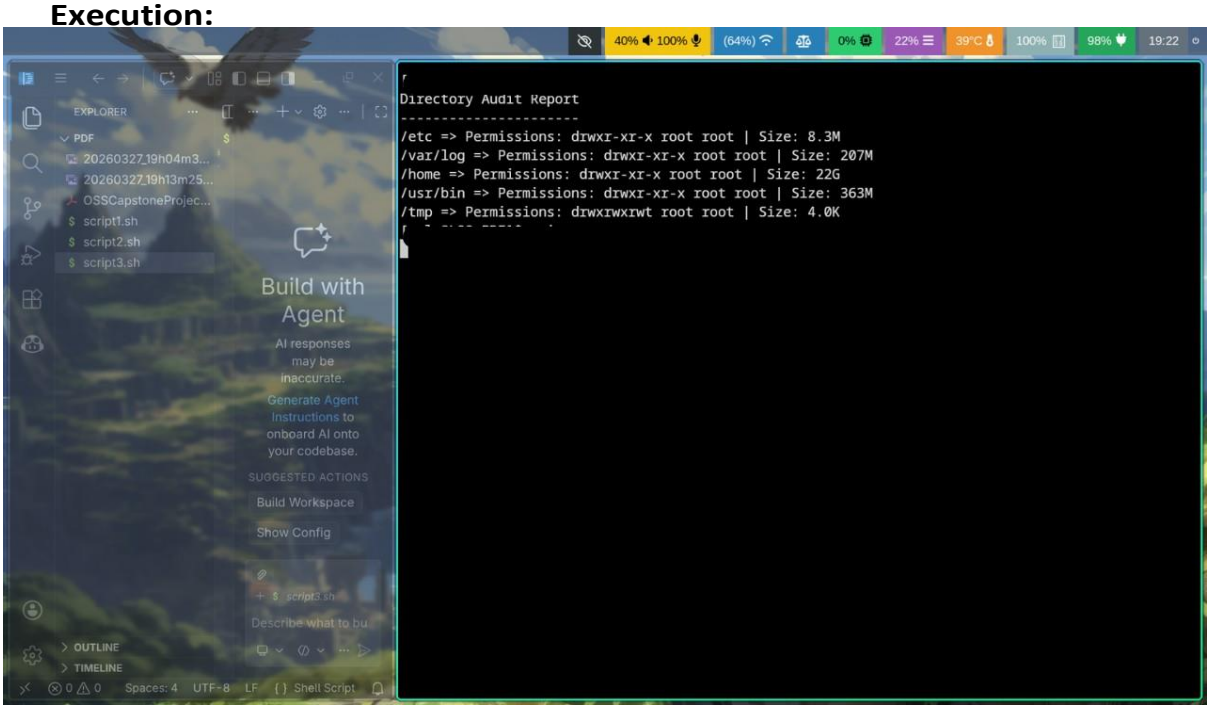


Explanation: This script check's if a package is installed if yes then it displays the package's info else it installs it into the system.

6.3 Script 3 — Disk and Permission Auditor

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
echo "Directory Audit Report"
echo " ---------------------------------- "
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "Nothing Found"
    fi
done
```

Execution:



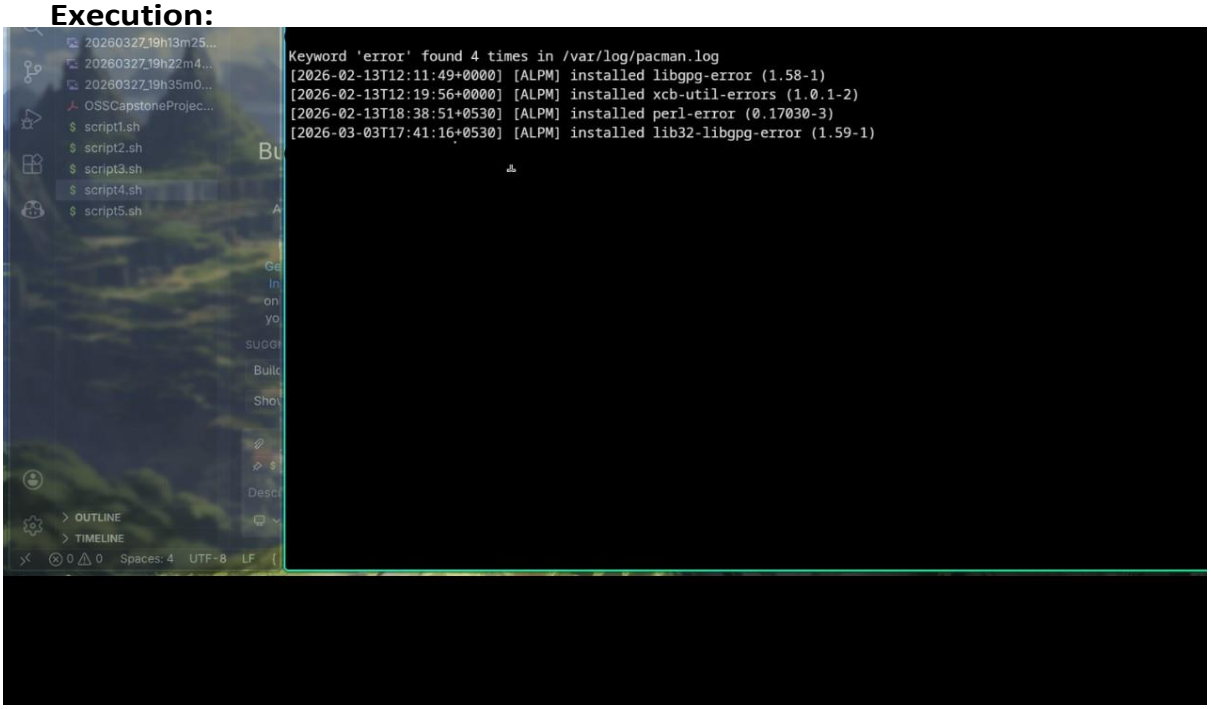
Explanation: This script cycles through the given directories and then prints out their permissions and directory sizes.

6.4 Script 4 — Log File Analyzer

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/messages [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"} # Default keyword is 'error'
COUNT=0
RETRIES=3

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Do-while style retry if file is empty
ATTEMPT=0
while true; do
    if [ -s "$LOGFILE" ]; then
        break
```

```
    fi
    ATTEMPT=$((ATTEMPT + 1))
    if [ "$ATTEMPT" -ge "$RETRIES" ]; then
        echo "Error: File $LOGFILE is empty after $RETRIES attempts."
        exit 1
    fi
    sleep 1
done

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# Print last 5 matching lines
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
```

Execution:

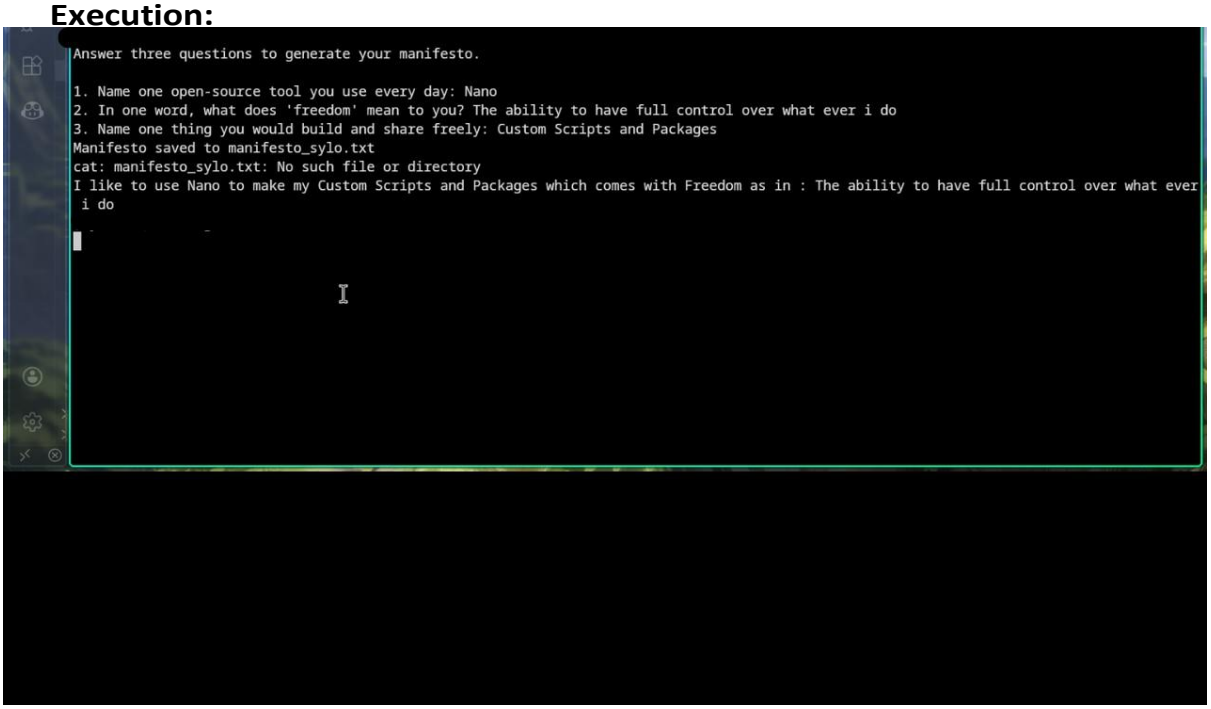


Explanation: Goes through the entire .log file and then prints out the results if they match , here we are searching for errors in pacman.log file , where Pacman is the Package manager for the ARCH-Based linux distros.

6.5 Script 5 — Manifesto Generator

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
echo "Answer three questions to generate your manifesto."
echo ""
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"
# TODO: Compose a paragraph using $TOOL, $FREEDOM, $BUILD
# and write it to $OUTPUT using echo and >>
echo "Manifesto saved to $OUTPUT"
cat $OUTPUT

echo "I like to use $TOOL to make my $BUILD which comes with Freedom as in : $FREEDOM
```

Execution:



Explanation: This Bash script is a simple interactive program that collects user input about open-source preferences and uses variables to store that data along with system information like the current date and username. It demonstrates basic shell scripting concepts such as input handling, variable assignment, and command substitution. However, while it prints a personalized statement to the terminal, it does not fully implement saving meaningful content to a file, making it more of a conceptual or partially completed script rather than a fully functional manifesto generator.

7 Conclusion

- The Linux kernel represents one of the most impactful achievements in the open-source world, demonstrating how collaborative development and transparent licensing can produce software that rivals and often surpasses proprietary alternatives. From its origins as a personal project by Linus Torvalds to becoming the backbone of modern computing systems, Linux highlights the power of community-driven innovation.

- Through this audit, it is evident that the FOSS ecosystem is not just about free software, but about freedom, control, and shared progress. The Linux kernel enables a wide range of technologies—from servers to mobile platforms—while maintaining flexibility and scalability. At the same time, its licensing model ensures that improvements remain accessible to everyone.

- In comparison to proprietary systems, Linux offers greater customization and control, though it may require more technical knowledge. However, continuous advancements and community efforts are steadily improving usability and adoption across all domains, including desktop computing and gaming.

- Overall, the Linux kernel and the broader open-source ecosystem illustrate a sustainable and forward-thinking approach to software development, where collaboration, transparency, and innovation take precedence over restriction and control.